```
If (M==0)
return heads
Else
return tails
```

We now have a service named TossService that contains a function coinFlip() that returns a string with value either 'heads' or 'tails'

Great! Lets save this file and now try to use it in our application.

```
@Component({
 selector: 'my-app',
template: `<h2>{{tossResult}}</h2>
providers: [TossService]})

export class TossComponent implements OnInit{

tossResult = 'Loading' ;
constructor TossComponent(private tossService : TossSer-
vice) {}

ngOnInit() {
this.tossResult = this.tossService.coinFlip();}}
```

As is evident from the example, this component uses the variable tossResult in its template that is initially set to the value ->'Loading'.

When the component is constructed, a DI for tossService private variable takes place and an instance of TossService is used. Finally, when ngOnInit is called (ngOnInit is a lifecycle hook for the component) the tossResult variable is set to either 'tails' or 'heads' based on the result of the function coinFlip() in TossService.

The result should be a div or html with the following output:

```
tails
```

Great work! We have finally created our very own service and used it in a component.

Complex angular applications have similarly designed (but of course more complex) services at their heart.

Services are responsible for delegation, bulk operations and all / most interactions with a back end server. Whenever designing services, do remember to use Promises/Observables to ensure that the service operates in a non-blocking manner.